



16 SEÇÕES

EXEMPLOS PRÁTICOS

COLINHA DE COMANDOS

Sumário

FUNDAMENTOS

- 1 O que é Git
- 2 Instalação e configuração
- 3 Os três estados do Git **ESSENCIAL**
- 4 Criando um repositório
- 5 O ciclo básico

TRABALHANDO DE VERDADE

- 6 Conectando com GitHub e GitLab
- 7 Boas mensagens de commit
- 8 O arquivo .gitignore
- 9 Branches
- 10 Merge
- 11 Resolvendo conflitos
- 12 Repositórios remotos

QUANDO DÁ ERRADO

- 13 Desfazendo coisas **ESSENCIAL**
- 14 Fluxo de trabalho na prática
- 15 Colinha de comandos
- 16 Erros comuns

DICA

As duas seções marcadas como **essenciais** são as que mais resolvem problema de iniciante. Se for ler só duas, leia essas.

Git é um sistema de controle de versão. Ele guarda o histórico de todas as alterações que você faz nos seus arquivos, permitindo:

- voltar a qualquer versão anterior;
- ver exatamente o que mudou, quando e por quem;
- trabalhar em várias funcionalidades ao mesmo tempo sem uma atrapalhar a outra;
- colaborar com outras pessoas sem sobrescrever o trabalho alheio.

Analogia: é como o histórico de versões de um documento, mas você decide quando salvar um marco e escreve uma anotação explicando o que fez.

Git não é GitHub

Confusão clássica, e vale gravar de vez:

O que é	
Git	O programa que roda na sua máquina e gerencia o histórico
GitHub, GitLab, Bitbucket	Sites onde você hospeda seus repositórios na nuvem

Git funciona 100% offline. O GitHub é apenas onde você guarda uma cópia online e compartilha com outras pessoas.

Vocabulário essencial

Termo	Significado
Repositório	A pasta do projeto rastreada pelo Git
Commit	Um ponto de salvamento no histórico, com mensagem
Branch	Uma linha de desenvolvimento paralela
Remote	O repositório hospedado (GitHub, GitLab)
Clone	Baixar um repositório remoto para sua máquina
Push	Enviar seus commits locais para o remoto
Pull	Trazer commits do remoto para sua máquina
Merge	Juntar duas branches

Instalando

LINUX (UBUNTU / DEBIAN)

```
sudo apt update && sudo apt install git
```

MACOS

```
brew install git
```

No **Windows**, baixe em git-scm.com — o instalador já inclui o Git Bash, um terminal com comandos Unix.

CONFIRME A INSTALAÇÃO

```
git --version
```

Configuração obrigatória

Todo commit registra quem foi o autor. Configure isso **uma vez** por máquina:

```
git config --global user.name "Seu Nome"
git config --global user.email "seu@email.com"
```

ATENÇÃO

Use o **mesmo e-mail** da sua conta do GitHub ou GitLab. É assim que os commits aparecem vinculados ao seu perfil.

Configurações que valem a pena

```
# branch inicial como 'main' (padrao moderno)
git config --global init.defaultBranch main

# seu editor de texto preferido
git config --global core.editor "code --wait"

# saida colorida
git config --global color.ui auto
```

VER TUDO O QUE ESTÁ CONFIGURADO

```
git config --list
```

3 Os três estados do Git essencial

Esse é o conceito que mais confunde iniciantes. Entender isso resolve metade dos problemas futuros. Um arquivo no Git pode estar em três lugares:

```
Working Directory      Staging Area      Repository
(sua pasta)           (area de preparo)  (historico)
|                     |                     |
|----- git add ----->|                     |
|                     |                     |
|                     |--- git commit -->|
```

Estado	O que significa
Working Directory	Seus arquivos como estão agora na pasta. Alterações aqui ainda não foram registradas.
Staging Area	Alterações selecionadas para entrar no próximo commit.
Repository (.git)	O histórico permanente. Commits ficam aqui.

Por que existe a Staging Area?

Ela permite **escolher o que entra em cada commit**. Se você alterou cinco arquivos mas apenas dois fazem parte de uma correção coerente, você adiciona só esses dois e faz um commit limpo.

O comando que mostra tudo

```
git status
```

DICA

Rode **git status** o tempo todo. Ele diz em que estado cada arquivo está e geralmente já sugere o próximo comando.

4 Criando um repositório

OPÇÃO A — COMEÇAR DO ZERO

```
mkdir meu-projeto
cd meu-projeto
git init
```

O **git init** cria uma pasta oculta **.git** — é ali que mora todo o histórico. Nunca apague nem edite essa pasta manualmente.

OPÇÃO B — CLONAR UM REPOSITÓRIO EXISTENTE

```
git clone https://github.com/usuario/repositorio.git
cd repositorio
```

O clone já traz o histórico completo e configura o remote automaticamente.

5 O ciclo básico

Esse é o fluxo que você vai repetir mil vezes:

```
editar arquivos -> git add -> git commit -> git push
```

Passo a passo

```
git status          # 1. o que mudou?
git add .           # 2. prepara
git commit -m "adiciona validacao" # 3. salva
git push            # 4. envia
```

VARIAÇÕES DO ADD

```
git add arquivo.java # um arquivo especifico
git add src/          # uma pasta inteira
git add .             # tudo que mudou
```

ATALHOS ÚTEIS

```
# add + commit (so para arquivos JA rastreados)
git commit -am "mensagem"

# ver o que mudou, linha por linha
git diff          # working dir vs staging
git diff --staged # staging vs ultimo commit

# historico
git log --oneline
git log --oneline --graph
```

Criando o repositório remoto

- Crie um repositório **vazio** no site;
- **não** marque a opção de inicializar com README se você já tem commits locais — isso cria histórico divergente;
- copie a URL fornecida.

Conectando o local ao remoto

```
git remote add origin https://github.com/usuario/repo.git
git branch -M main
git push -u origin main
```

Comando	O que faz
remote add origin	Registra a URL sob o apelido origin (convenção universal)
branch -M main	Renomeia a branch atual para main
push -u origin main	Envia e vincula a branch local à remota. O -u faz com que dali em diante baste git push

HTTPS ou SSH?

	HTTPS	SSH
URL	https://github.com/user/repo.git	git@github.com:user/repo.git
Autenticação	Personal Access Token	Chave SSH
Configuração	Mais simples no início	Configura uma vez, nunca mais pede senha

ATENÇÃO

Desde 2021 o GitHub não aceita mais a senha da conta via HTTPS — você precisa de um **Personal Access Token**. É por isso que o SSH acaba sendo mais confortável a médio prazo.

CONFIGURANDO SSH

```
# 1. gerar a chave
ssh-keygen -t ed25519 -C "seu@email.com"

# 2. copiar a chave publica
cat ~/.ssh/id_ed25519.pub
```

Cole o conteúdo em **GitHub** → **Settings** → **SSH and GPG keys** → **New SSH key**. Depois teste:

```
ssh -T git@github.com
```

7 Boas mensagens de commit

Uma mensagem ruim hoje é um problema seu daqui a três meses.

Regras práticas

- **Modo imperativo:** “adiciona X”, não “adicionado X” ou “adicionei X”;
- **primeira linha até cerca de 50 caracteres**, resumindo a mudança;
- descreva o **quê** e o **porquê**, não o *como* — o código já mostra o como;
- **um commit = uma mudança lógica**. Não misture correção de bug com refatoração.

RUIM

```
update
correcoes
asdasd
mexi no arquivo
```

BOM

```
corrige calculo de media que ignorava o peso
adiciona validacao de CPF no cadastro
remove dependencia nao utilizada do pom.xml
```

Conventional Commits

Padrão muito usado em empresas. Um prefixo indica o tipo da mudança:

Prefixo	Usado para
feat:	Nova funcionalidade
fix:	Correção de bug
docs:	Documentação
refactor:	Refatoração sem mudança de comportamento
test:	Testes
chore:	Manutenção, dependências, configuração

```
feat: adiciona endpoint de busca por artista
fix: corrige divisao inteira no calculo de media
docs: atualiza README com instrucoes de instalacao
```


Para escrever uma mensagem longa com título e corpo, rode **git commit** sem o **-m**: o Git abre o editor configurado.

8 O arquivo .gitignore

Nem tudo deve ir para o repositório. Crie um arquivo chamado **.gitignore** na raiz do projeto:

```
# Dependencias
node_modules/
target/
build/

# Ambiente e SEGREDOS
.env
application-local.properties
credenciais.txt

# Compilados
*.class
*.jar
*.log

# IDE
.idea/
.vscode/

# Sistema operacional
.DS_Store
Thumbs.db
```

O erro mais caro de iniciante

ATENÇÃO

Nunca commite senhas, tokens ou credenciais. Uma vez no histórico, elas ficam lá mesmo depois de você apagar o arquivo — qualquer pessoa com acesso ao repositório recupera com `git log`. Se acontecer, **troque a credencial imediatamente**. Remover do histórico é possível, mas trabalhoso, e não desfaz o vazamento se o repositório for público.

O site **gitignore.io** gera o arquivo pronto: você informa a tecnologia (Java, Node, Python) e ele monta a lista.

Arquivo já commitado que deveria estar ignorado

```
git rm --cached arquivo.env
```

Isso remove do rastreamento **sem apagar do seu disco**. Depois adicione ao `.gitignore` e faça commit.

Uma **branch** é uma linha de desenvolvimento independente. Você trabalha nela sem afetar a principal. Isso permite desenvolver algo novo sem quebrar o que já funciona — e se der errado, basta descartar a branch.

```
main      A---B---C-----G
           \         /
feature    D---E-F
```

Comandos

```
git branch          # listar (a atual tem *)
git branch nome     # criar
git checkout nome   # trocar
git switch nome     # trocar (moderno)

git checkout -b nome # criar E trocar
git switch -c nome   # idem, moderno

git branch -d nome   # apagar (apos merge)
git branch -D nome   # forçar

git push -u origin nome # enviar branch nova
```

NOMES POR CONVENÇÃO

```
feature/busca-por-artista
fix/calculo-media
docs/readme
refactor/camada-service
```

Merge é trazer as alterações de uma branch para outra.

```
git checkout main      # 1. branch que vai RECEBER
git merge feature/busca # 2. traz as mudancas
git push               # 3. envia
git branch -d feature/busca # 4. apaga a branch mesclada
```

Fast-forward e merge commit

- **Fast-forward** — se a main não mudou desde que você criou a branch, o Git apenas avança o ponteiro. Histórico linear, sem commit extra;
- **merge commit** — se ambas mudaram, o Git cria um commit de junção com dois pais.

Pull Request e Merge Request

Em equipes, você geralmente não faz merge direto. Abre um **Pull Request** (GitHub) ou **Merge Request** (GitLab) pelo site, alguém revisa, e o merge acontece por lá. O fluxo local é o mesmo até o git push da branch.

11 Resolvendo conflitos

Conflito acontece quando duas branches alteram **a mesma linha** do mesmo arquivo. O Git não sabe qual versão manter e pede sua decisão.

COMO APARECE NO ARQUIVO

```
<<<<<<< HEAD
    double media = (a + b) / 2.0;
=====
    double media = (a * 2 + b * 3) / 5.0;
>>>>>> feature/media-ponderada
```

Marcador	Significado
<<<<<<< HEAD	Início da versão da branch atual
=====	Separador entre as duas versões
>>>>>> branch	Fim da versão que está entrando

Como resolver

- Abra o arquivo;
- **decida** o que fica: uma das versões, as duas, ou algo novo;
- **apague os marcadores** de conflito;
- salve o arquivo;
- marque como resolvido e finalize.

```
git add arquivo-resolvido.java
git commit
```

COMANDOS DE APOIO

```
git status          # quais arquivos estão em conflito
git merge --abort    # cancela e volta ao estado anterior
```

DICA

O VS Code destaca conflitos visualmente e oferece botões *Accept Current*, *Accept Incoming* e *Accept Both*. Bem mais confortável que editar na mão.

12 Repositórios remotos

```
git fetch          # baixa MAS nao aplica
git pull           # fetch + merge (baixa e aplica)
git pull --rebase  # reaplica seus commits por cima
```

Diferença importante: o fetch é seguro, apenas informa o que existe no remoto. O pull altera seus arquivos.

DICA

Sempre dê git pull antes de começar a trabalhar, especialmente em equipe. Esse único hábito evita a maior parte dos conflitos.

13 Desfazendo coisas essencial

Essa é a seção que você vai voltar a consultar.

Descartar alterações não commitadas

```
git restore arquivo.java  # descarta mudancas num arquivo
git restore .              # descarta TUDO nao commitado

# equivalente antigo:
git checkout -- arquivo.java
```

ATENÇÃO

O **git restore** apaga seu trabalho não salvo. Não tem volta.

Tirar do staging mantendo as alterações

```
git restore --staged arquivo.java
git reset arquivo.java # antigo
```

Corrigir o último commit

```
# esqueceu um arquivo
git add arquivo-esquecido.java
git commit --amend --no-edit

# so trocar a mensagem
git commit --amend -m "mensagem corrigida"
```

ATENÇÃO

Nunca use `--amend` em commit que já foi enviado. Ele reescreve o histórico e quebra a sincronia com quem já baixou.

Desfazer commits

Comando	Commit	Alterações
<code>git reset --soft HEAD~1</code>	Desfeito	Preservadas, no staging
<code>git reset HEAD~1</code>	Desfeito	Preservadas, fora do staging
<code>git reset --hard HEAD~1</code>	Desfeito	Apagadas

Desfazer um commit já publicado

```
git revert HASH_DO_COMMIT
```

O **revert** cria um **novo commit** que desfaz o anterior. Não reescreve o histórico, então é seguro para commits já enviados.

Guardar trabalho temporariamente

```
git stash      # guarda e limpa o working directory
git stash list # lista o que esta guardado
git stash pop  # recupera o mais recente e remove da pilha
git stash apply # recupera mas mantem na pilha
git stash drop # descarta
```

Muito útil quando você precisa trocar de branch no meio de algo inacabado.

A rede de segurança

```
git reflog
```

Mostra **tudo** o que aconteceu no repositório, incluindo commits que você “perdeu” com um `reset --hard`. Se fez besteira, o hash provavelmente está aí:

```
git reset --hard HASH_QUE_APARECEU_NO_REFLOG
```

DICA

Grave o **git reflog**. Ele recupera quase tudo que você achou que tinha perdido para sempre.

14 Fluxo de trabalho na prática

Trabalhando sozinho

```
git pull                                # comece atualizado
# ... edita arquivos ...
git status                             # confere o que mudou
git add .
git commit -m "feat: adiciona busca"
git push
```

Trabalhando com branches

```
git checkout main
git pull                                # main atualizada

git checkout -b feature/nova-tela
# ... desenvolve ...
git add .
git commit -m "feat: cria tela de detalhe"
git push -u origin feature/nova-tela

# abre o Pull/Merge Request no site
# depois de aprovado e mesclado:
git checkout main
git pull
git branch -d feature/nova-tela
```

Momento	Comando
Antes de começar	git pull
Antes de commitar	git status e git diff
Ao terminar um pedaço lógico	git add e git commit
Antes de sair	git push

CONFIGURAÇÃO

```
git config --global user.name "Nome"
git config --global user.email "email@exemplo.com"
git config --list
```

COMEÇANDO

```
git init          # cria repositório
git clone URL      # baixa repositório existente
```

DIA A DIA

```
git status          # o que mudou
git add .           # prepara tudo
git add arquivo     # prepara um arquivo
git commit -m "msg" # salva
git push           # envia
git pull           # baixa
git log --oneline   # histórico resumido
git diff           # mudanças nas linhas
```

BRANCHES

```
git branch          # lista
git checkout -b nome # cria e entra
git checkout nome    # troca
git merge nome       # mescla
git branch -d nome   # apaga
```

DESLAZENDO

```
git restore arquivo      # descarta alterações
git restore --staged arq  # tira do staging
git commit --amend        # corrige último commit
git reset --soft HEAD~1   # desfaz commit
git revert HASH           # desfaz publicado
git stash                 # guarda temporariamente
git reflog                # salva-vidas
```

16 Erros comuns

`fatal: not a git repository`

Você não está dentro de uma pasta com Git. Verifique com **pwd** e navegue até o projeto, ou rode **git init**.

`Your branch is ahead of 'origin/main' by N commits`

Você tem commits locais que ainda não foram enviados. Rode **git push**.

`Updates were rejected because the remote contains work you do not have`

Alguém enviou algo que você não tem. Rode **git pull** primeiro, resolva eventuais conflitos, e só então **git push**.

`Please tell me who you are`

Falta configurar nome e e-mail. Veja a seção 2.

`Support for password authentication was removed`

O GitHub não aceita mais senha via HTTPS. Use um Personal Access Token ou configure SSH (seção 6).

Comitei no branch errado

```
git reset --soft HEAD~1 # desfaz o commit, mantém alterações
git stash                # guarda
git checkout branch-correta
git stash pop            # recupera
git add . && git commit -m "mensagem"
```

Comitei uma senha por engano

- Troque a credencial imediatamente — assumo que ela vazou;
- adicione o arquivo ao `.gitignore`;
- rode **git rm --cached arquivo**;
- faça commit e push;
- para remover do histórico (avançado): `git filter-repo` ou `BFG Repo-Cleaner`.

detached HEAD

Você está num commit específico, não numa branch.


```
git checkout main          # sair
git branch nova-branch     # ou salvar o que voce fez ali
git checkout nova-branch
```

Para continuar aprendendo

Recurso	Endereço
Documentação oficial	git-scm.com/doc
Livro Pro Git (gratuito, em português)	git-scm.com/book/pt-br/v2
Prática visual e interativa de branches	learngitbranching.js.org

Além disso, crie um repositório de teste e quebre à vontade. É a melhor forma de perder o medo dos comandos de desfazer — e a única maneira de descobrir que quase nada é irreversível.

Resumo em cinco frases

- 1 **git status é seu melhor amigo** — rode sempre que estiver na dúvida.
- 2 **Commits pequenos e frequentes** valem mais que um commit gigante no fim do dia.
- 3 **git pull antes de começar** evita a maioria dos conflitos.
- 4 **Nunca commite senhas** — e se commitar, troque a senha; não confie em apagar.
- 5 **git reflog recupera** quase tudo que você achou que perdeu.